

LLM-guided decision-making toolkit for multi-agent reinforcement learning

Zhemín Li¹, Ruobing Zhang, Zhengming Wang, Zheng Xie, Yiping Song^{*}

College of Science, National University of Defense Technology, Changsha, 410000, China

ARTICLE INFO

Communicated by P. Li

Keywords:

MARL
LLM
SMAC
Cold-start problem

ABSTRACT

Complex cooperative game tasks pose significant challenges to decision making in multi-agent systems. Despite advancements in multi-agent reinforcement learning (MARL), these systems often struggle in the initial learning phase, known as the cold-start problem, where devising effective strategies from scratch is difficult. To address this, we leverage the advanced reasoning capabilities of Large Language Models (LLMs) by introducing prompt learning in complex cooperative games. In our approach, tensor-formatted environmental information is converted into natural language and fed into LLMs during early exploration stages in MARL algorithms. The strategic suggestions generated by LLMs, combined with trajectories from random exploration, form an experience pool for initial training. Our experiments demonstrate that this approach significantly enhances cooperation among agents in complex games and mitigates the cold-start issue. Notably, we have developed a toolkit compatible with the StarCraft Multi-Agent Challenge (SMAC) environment, providing seamless integration between SMAC APIs, LLMs, and MARL algorithms. This includes automated prompt generation for any map and alignment of natural language strategies with tensor-formatted data. In addition, considering the importance of effective communication for enhancing collaboration efficiency, our toolkit includes a robust communication module, which supports the training, testing, and visualization of communication algorithms. We provide the source code on GitHub at https://github.com/lizhemin18/pymarl_LLM.

1. Introduction

In recent years, complex cooperative game tasks have emerged as a significant focus in multi-agent systems [1]. These tasks typically involve dynamic and uncertain environments, requiring coordination and decision making between agents [2]. Multi-agent reinforcement learning (MARL) has become a powerful framework that allows agents to learn optimal strategies through interactions with the environment and other agents [3].

Despite remarkable progress in MARL, several challenges remain. One critical issue is the cold start problem [2,4]. During the cold start phase, agents explore the environment from an unknown state due to lack of initial experience, leading to inefficient behaviors early on. Sparse reward signals complicate learning effective strategies [5,6], while high-dimensional state spaces non-stationary environments exacerbate the challenge, as past experiences are difficult to transfer to new situations [7,8]. The cold start problem is particularly complex in MARL due to the interdependence between agents [2,4]. The learning process becomes unstable as agents rely on each other, and a lack of initial experience makes it difficult to [2]. Moreover, the non-stationarity of policies implies that changes in one agent's strategy affect the overall dynamics, making it difficult to evaluate the game situation as a whole [7]. Furthermore, the ambiguity of reward assignment makes

it difficult for agents to discern their contribution to the reward [4,9]. These factors collectively intensify the cold start challenge in MARL.

Traditional approaches to solving the cold start problem typically involve the use of pre-trained models, imitation learning, and leveraging expert knowledge [10–12]. These methods can quickly provide some baseline behavioral patterns, reducing the time agents spend blindly exploring the environment [10,11]. However, they also have significant limitations. Pre-trained models may not generalize well to specific task environments [10], imitation learning relies on high-quality demonstration data [11], and expert knowledge may introduce subjective biases and is often difficult to scale [12]. Furthermore, these methods typically require large amounts of labeled data, adding to the complexity and cost of implementation [10,11].

Large language models (LLMs) offer an effective solution to address these limitations [13,14]. With the vast knowledge base, experience, and powerful reasoning capabilities, LLMs can generate contextually relevant strategy recommendations without relying on labeled data [14]. This makes LLMs an ideal tool for alleviating the cold start problem in MARL, particularly in helping agents rapidly adapt to complex cooperative game tasks [13]. Using the strengths of LLMs, it is possible to significantly improve learning efficiency and collaborative

^{*} Corresponding author.

E-mail address: songyiping@nudt.edu.cn (Y. Song).

effectiveness in situations where initial experience is lacking. However, integrating LLMs into MARL frameworks presents unique challenges. In StarCraft Multi-Agent Challenge (SMAC) [15–17], where conditions change rapidly from frame to frame, each moment introduces new uncertainties and complexities. The high dimensionality of state and action spaces in multi-agent systems further complicates this integration, making it difficult for LLMs to directly interface with SMAC.

In this paper, we explore the integration of LLMs into MARL frameworks to enhance decision-making during the exploration phase of multi-agent systems. By leveraging LLMs to assist in decision-making, we aim to improve the training efficiency of MARL algorithms and address the cold start challenge. This approach highlights the potential of LLMs in boosting MARL efficiency and opens new research avenues in multi-agent systems, especially in reducing reliance on labeled data for supervised learning.

Additionally, we present a comprehensive toolkit that supports automatic prompt generation for any map at any moment in SMAC, integration with multiple LLMs, automatic strategy conversion, and LLM-assisted MARL algorithm training, creating a complete end-to-end workflow. Given the extensive research on communication in MARL, the toolkit also offers functionalities for training and testing communication algorithms in real-world multi-agent communication and co-operation scenarios. It includes features for visualizing communication processes to aid in algorithm analysis.

Our contributions are threefold:

(1) To address the cold start challenge in multi-agent reinforcement learning (MARL), we leverage large language models (LLMs) to provide strategic guidance in complex cooperative game environments, thereby mitigating the cold start problem.

(2) We develop a multi-functional toolkit integrated with the API of the SMAC environment, supporting dynamic prompt generation, automatic strategy conversion, LLMs integration, and visualization, thus advancing the research capabilities of MARL tasks.

(3) For real-world multi-agent communication and cooperation scenarios, we embed robust communication algorithm training and testing functionalities within the toolkit and facilitate algorithm analysis through communication visualization features.

2. Related work

2.1. Multi-agent reinforcement learning

Multi-agent reinforcement learning (MARL) involves multiple agents learning through interactions within a shared environment, optimizing their decisions based on rewards and the actions of others. Recent developments in MARL have introduced several algorithms that address cooperation and competition among agents.

Value Decomposition Networks (VDN) [18] simplify cooperation by breaking down the joint value function into individual components. Counterfactual Multi-Agent Policy Gradient (COMA) [19] enhances cooperative behavior using counterfactual reasoning. Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [2] extends actor-critic methods to mixed environments. Asynchronous Advantage Actor-Critic (A3C) [20] improves learning by training agents in parallel. QTRAN [21] and Qatten [22] refine Q-value decomposition to handle complex coordination. Dueling DQN [23] separates state evaluation from action evaluation, and BiCNet [24] uses recurrent networks for better agent communication.

Algorithms like QMIX [25] use nonlinear networks to combine Q-values for effective collaboration, while Independent Proximal Policy Optimization (IPPO) [26] allows agents to optimize independently, making it adaptable to environments without global state information. These two algorithms are noted for their stability, making them suitable baselines for assessing the impact of global state information on MARL performance.

Despite the advancements, a common challenge remains: many methods start with random initialization, causing a cold start where agents must learn from scratch. This slows early learning and overall optimization, highlighting the need for strategies that can overcome the initial exploration hurdle.

2.2. Cold-start problem in multi-agent reinforcement learning

In multi-agent reinforcement learning (MARL), the cold start problem arises when agents struggle to initialize their policies effectively due to a lack of prior experience, leading to slow or suboptimal learning. To address this, techniques such as experience replay have been employed to improve data efficiency by learning from diverse past experiences. The Ape-X framework [27] achieves state-of-the-art performance by distributing data generation and prioritizing informative experiences. Experience replay is stabilized by [28] to manage the non-stationarity caused by independent Q-learning, enhancing early-stage learning.

Another approach involves effectively initializing policies to provide agents with a strong starting point, reducing convergence time. Curiosity-driven exploration [6] uses intrinsic rewards to encourage exploration even in sparse reward environments, while the rl-imgep framework [29] allows agents to explore autonomously using intrinsic motivation without predefined goals, helping them learn despite initial inexperience.

Cooperation strategies in MARL also mitigate cold starts. Multi-agent policy gradient algorithms with centralized critics [2] and communication scheduling methods like SchedNet [30] enable efficient learning by improving communication and reducing variance in gradient estimates. The ATOC framework [31] and IC3Net model [32] optimize communication by allowing agents to selectively share information or decide when to communicate, focusing on essential interactions. VBC [33] prioritizes communication when agents are uncertain, enhancing learning efficiency.

While these methods address the cold start problem to varying degrees, they still depend on random initialization, which limits early-stage performance and can hinder scalability to complex environments. Thus, there remains a significant need for more robust and scalable cold start solutions that balance effective initialization, coordination, and adaptability to diverse MARL scenarios.

2.3. LLM-enhanced multi-agent reinforcement learning

The integration of Large Language Models (LLMs) has led to notable advancements in fields such as NLP, code generation, healthcare, and multimodal tasks. Radford et al. [34] introduced GPT-2, which demonstrated the effectiveness of large-scale unsupervised pretraining for tasks like translation, summarization, and question-answering, establishing LLMs as versatile multitask learners. In code generation, Chen et al. [35] developed models that led to GitHub Copilot, significantly improving programming efficiency by generating code from natural language prompts. Wang et al. [36] introduced ClinicalGPT in healthcare, enhancing clinical data analysis and supporting medical decision-making. Gao et al. [37] showcased GPT-3's ability in few-shot learning for question-answering, reducing the dependency on extensive task-specific training.

Simultaneously, LLMs are being integrated into MARL systems to enhance collaboration and decision-making. Li et al. [38] explored LLMs for Theory of Mind in multi-agent collaborations, demonstrating advanced reasoning but noting challenges in long-term planning. Yao et al. [39] introduced the VELO framework, optimizing Quality of Service in cloud-edge systems by integrating LLMs with MARL algorithms. Sun et al. [40] reviewed LLMs-based MARL frameworks, highlighting their potential for tasks requiring communication and shared goals, and suggesting future research directions. D. Morad et al. [41]

proposed a language-conditioned offline reinforcement learning approach for multi-robot navigation, where LLMs help robots interpret and generalize natural language instructions effectively.

Despite the promising potential of LLMs in MARL fields, existing studies often fail to fully integrate LLMs with the MARL algorithms in the SMAC environment, particularly lacking the flexibility to handle random map setups. This restricts their application across diverse scenarios. Moreover, communication is a critical component in multi-agent systems, yet there is limited support for communication algorithms and visualization tools within the SMAC environment. This deficiency impedes deeper insight and optimization of communication strategies among agents.

Addressing these challenges is essential for advancing the field. Improving integration with the MARL workflow, ensuring adaptability to any map configuration, and offering robust communication algorithms and visualization tools can significantly enhance experimental flexibility and efficiency. Such improvements are crucial for better understanding complex interactions in multi-agent systems and optimizing overall performance.

3. Multi agent reinforcement learning based on LLM-assisted decision making

In this section, we present an algorithm that improves MARL with the assistance of LLMs. The LLMs provide strategic suggestions during early training, reducing the reliance on purely random exploration. Section 3.1 outlines the problem and sets the notation. Section 3.2 explains the process of generating prompts for the LLMs and converting their outputs into actionable strategies for the agents in the StarCraft Multi-Agent Challenge (SMAC) environment. Section 3.3 then details how LLM-generated suggestions are integrated into the MARL process to improve both exploration and strategy development over time.

3.1. Problem definition

In a collaborative combat setting, we introduce a MARL challenge where a team of n allied agents confronts m enemy agents. The agents' actions are orchestrated by an algorithm, which receives feedback in the form of rewards from the environment following each action. This feedback loop allows the agents to continuously improve their strategies, with the ultimate goal of neutralizing the enemies to win the game. Our objective is to optimize the training phase so that the most effective strategy can be identified swiftly, thereby minimizing the time wasted on unproductive exploration.

We use decentralized partially observable Markov decision processes (Dec-POMDPs) as a modeling framework for multi-agent decision making. A Dec-POMDP is formally represented as a tuple $(I, S, A, T, O, R, \gamma)$, where I denotes the finite set of agents, S the set of states, A^i the action set for agent i , and T the state transition function. Furthermore, O^i is the observation set for agent i , R is the reward function, and γ is the discount factor.

At each time step t , each agent i chooses an action $a_t^i \in \mathcal{A}$ based on its own observations o_t^i . The collective action of all agents, $a_t = \{a_t^1, \dots, a_t^n\}$, determines the next state of the environment s_{t+1} and the immediate rewards r_t^i for each agent, as determined by the reward function $R : S \times \mathcal{A} \rightarrow \mathbb{R}$. Working together to accomplish cooperative tasks, agents aim to find policies $\{\pi^1, \dots, \pi^n\}$ that maximize their overall team utility.

Since our goal is to leverage LLMs to generate empirical initial game strategies in the early stages of the training algorithm, we adopt a dual-perspective approach to decision making, including decentralized and centralized perspectives, tailored to different scenarios.

In the decentralized perspective, each agent independently feeds its local observation o_t^i to the LLM at each time step t . The LLM then generates an action a_t^i based solely on the individual observation of the agent. This decentralized method allows agents to develop localized

strategies, which can be particularly effective in environments where agents have limited or partial information about the global state. It is suitable for scenarios where individual autonomy and adaptability are crucial.

In the centralized perspective, we feed the LLM with the global environmental state s_t at each time step t . In this approach, the LLM produces a coordinated action plan for all agents. This method facilitates more strategic and cooperative behavior, especially in scenarios that require synchronized actions among multiple agents. It is ideal for environments where global coordination and centralized control are necessary.

In both cases, the interaction between the LLM and agents is bidirectional. At each time step t , we input the global state s_t (centralized) or each agent's local observation o_t^i (decentralized) into the LLM to generate actions a_t^i . These interactions result in the execution of the action, yielding a return r_t and the next state s_{t+1} . These interactions are recorded as trajectories $\langle s_t, a_t, r_t, s_{t+1} \rangle$ and stored in the replay buffer.

3.2. Bi-direction interaction between LLMs and SMAC environment

In this section, we explore the bi-directional interaction LLMs and SMAC environment in a multi-agent system, including the mechanisms for generating prompts and interpreting the resulting strategies to guide agent behavior.

3.2.1. Prompt transformation

At each time step t , the state s_t and each agent's observation o_t^i are formulated into natural language descriptions. The prompt transformation process consists of four parts:

1. **Prompt Generation Supportive Information Repository:** Prior to generating prompts, we create an information repository to support the prompt generation process, which is included in the SMAC package. Once the training map (including custom maps) is determined, this repository aids in generating specific prompt information. The repository contains configurations of agents from both sides on the map, detailing agent attributes such as health, defense, vision and shooting range. This allows LLMs to gain a thorough understanding of both the environment and the specific characteristics of the agents.
2. **System Prompt Development:** The system prompt contains static information that remains constant throughout the training. It is generated based on the current training map and the attributes of the agents involved. This includes essential details like the environmental state and the agents' fixed properties, such as health, shield, cooldown, and offensive capabilities. The system prompt provides a stable and comprehensive context for the LLMs, ensuring they have a consistent understanding of the environment and agent configurations. Consider the transformation of a prompt at a specific time step on a particular map in the StarCraft Multi-Agent Challenge (SMAC). The centralized system prompt is illustrated in Fig. 1(a), whereas the decentralized system prompt is depicted in Fig. 2(a).
3. **User Prompt Development:** The user prompt captures dynamic information that changes at each time step. In the centralized approach, as shown in Fig. 1(b), the user prompt is derived from the global state s_t in the time step t . It includes detailed information about all allied and enemy agents, regardless of individual agent visibility. This ensures that the LLM has a complete view of the environment. In contrast, in the decentralized approach, as shown in Fig. 2(b), the user prompt is generated based on each agent's local observation o_t^i . It includes information about the agent's own information, as well as details about any allied and enemy agents within its visual range. This localized perspective allows each agent to make decisions based on its immediate surroundings. In both approaches, the available actions for each agent remain consistent, ensuring uniformity in the decision-making process.

(a) System Prompt	(b) User Prompt
### Map and Objective - Map: '3s5z_vs_3s6z' - 32.0 * 32.0 units. - Teams: Allies (West) vs Enemies (East). - Allies Composition: 3 stalkers and 5 zealots. - Enemies Composition: 6 zealots and 3 stalkers. - Objective: Coordinate to eliminate enemy agents. ### Unit Descriptions - Stalkers start with 80 health and 80 shield, each attack deals 13 damage. - Zealots begin with 100 health and 50 shield, each attack deals 8 damage. - Protoss units (stalkers, zealots, colossus) must have their shields down before health can be damaged. ### Game Mechanics - Observation Range: 9 units. - Attack Range: 6 units. - Movement: Agents can move 2 units in any cardinal direction per time step. - Shields: Protoss units have additional shields that must be exhausted before health can be targeted. ### Coordination Guidelines - Target Allocation: Assign only the needed allies to an enemy, unless a strategic advantage calls for more. - Communication: Allies should execute coordinated actions. - Efficiency: Aim for effective attacks, reduce missed shots, and move allies strategically for maximum impact. ### Response Format - Actions are encoded as 15-bit binary vectors with a single '1' indicating the selected action. Response should be a list of eight such vectors, formatted within square brackets, comma-separated. - For instance, if the first two agents' optimal actions are moving east by 2 units and attacking the first enemy, the vectors would be: [0,0,0,0,1,0,0,0,0,0,0,0,0,0,0], [0,0,0,0,0,0,1,0,0,0,0,0,0,0,0].	- Please provide the optimal actions for the 8 allied agents according to the current game state, ensuring they work together for the best possible outcome. The actions should follow the coordination guidelines outlined in the system prompt. ### Current Game State ### Ally Status - 1st Ally: stalker, Position: (-1.09, -1.22), Health: 80.00, Shield: 80.00, Cooldown: 0.45 - 8th Ally: zealot, Position: (-3.67, 1.08), Health: 100.00, Shield: 50.00, Cooldown: 0.00 ### Enemy Status - 1st Enemy: stalker, Position: (4.24, 0.15), Health: 80.00, Shield: 80.00 - 9th Enemy: stalker, Position: (4.10, -1.66), Health: 80.00, Shield: 80.00 ### Action Selection - For each of the 8 allied agents, select one action from the following options. 'N/A' denotes unavailable. - For the 1st allied agent: - 1. No-operation (if dead) - 2. Stay stationary - 3. Move 2 units north - 7. Attack the 1st enemy. - 15. Attack the 9th enemy. - For the 2nd allied agent: - For the 9th allied agent:

Fig. 1. Global state prompt in 3s5z_vs_3s6z map at a timestep (Centralized perspective).

4. Final Prompt Synthesis: The system prompt and the user prompt together form a complete input to the LLMs. This integration ensures that the generated strategies reflect both the overall context (from the system prompt) and the current situation (from the user prompt). By combining these elements, the LLMs can generate more accurate and applicable strategies, thereby improving the performance of the multi-agent system.

3.2.2. Policy generation

When we input prompts into LLMs, the models generate policy suggestions in natural language. These suggestions must then be translated into actions that the agents can execute in SMAC environment. The process of generating these actions differs between decentralized and centralized approaches, each with its own advantages and challenges.

1. Centralized Policy Generation: In the centralized approach, the LLM receives the global environmental state s_t at each time step t . It then generates a set of coordinated actions a_t for

all agents simultaneously. This process enables the LLM to derive collaborative strategies through comprehensive analysis of inter-agent relationships and agent-specific attributes.

As shown in Fig. 3, when Allied Unit 1 selects Enemy 3 as the target, Allied Unit 2 coordinates to focus fire on the critically wounded enemy, while Allied Unit 3 strategically retreats due to its vulnerable defense status. This method allows the LLM to consider the entire environment, ensuring that the actions are well coordinated and optimized for collective performance. However, processing the full state requires more computational resources, as the prompt contains more information (tokens). Despite this, the centralized approach requires only one LLM call per time step, making it efficient in terms of call volume.

2. Decentralized Policy Generation: In the decentralized approach, the LLM processes the local observation o_t^i of each agent i separately at time step t . For each agent i , the LLM generates an individual action a_t^i based on the prompt of o_t^i . These

(a) System Prompt	(b) User Prompt
### Map and Objective - Map: '3s5z_vs_3s6z' - 32.0 * 32.0 units. - Teams: Allies (West) vs Enemies (East). - Allies Composition: 3 stalkers and 5 zealots. - Enemies Composition: 6 zealots and 3 stalkers. - Objective: Coordinate with allied agents to eliminate enemy agents based on current local observations. ### Unit Descriptions - Stalkers start with 80 health and 80 shield, each attack deals 13 damage. - Zealots begin with 100 health and 50 shield, each attack deals 8 damage. - Protoss units (stalkers, zealots, colossus) must have their shields down before health can be damaged. ### Game Mechanics - Observation Range: 9 units. - Attack Range: 6 units. - Movement: Agents can move 2 units in any cardinal direction per time step. - Shields: Protoss units have additional shields that must be exhausted before health can be targeted. ### Response Format - Select the action as a 15-bit binary vector, with a single '1' indicating the selected action. - Example 01: Move 2 units east: [0,0,0,0,1,0,0,0,0,0,0,0,0,0,0] - Example 02: Attack 1st enemy: [0,0,0,0,0,0,1,0,0,0,0,0,0,0,0]	- Based on the current situation of my unit, please provide the most optimal action recommendation. ### Current Game State ### My Unit - Type: zealot, Health: 100.0/100, Shield: 50.0/50 - Position: (12.07, 17.01) ### Ally Status - 1st Ally: stalker, Relative coordinates (-1.85, -2.29), Health: 80.00, Shield: 80.00, distance:2.9 - 8th Ally: dead or out of view. ### Enemy Status - 1st Enemy: stalker, Relative coordinates (2.64, 2.32), Health: 80.00, Shield: 80.00, distance:3.5 - 9th Enemy: stalker, Relative coordinates (5.28, -2.87), Health: 80.00, Shield: 80.00, distance:6.0 ### Action Selection - For 'my unit' elect one action from the following options. 'N/A' denotes unavailable. - 1. Not Available (N/A) - 2. Stay stationary - 3. Move 2 units north - 7. Attack the 1st enemy - 15. Not Available (N/A)

Fig. 2. Local observation prompt in 3s5z_vs_3s6z map for an agent at a timestep (Decentralized perspective).

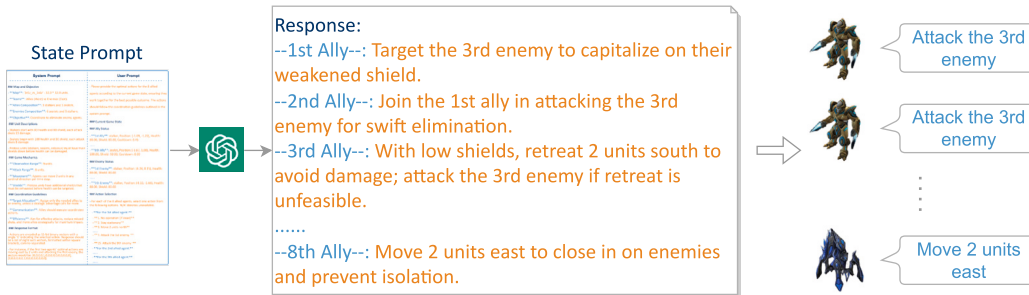


Fig. 3. Action recommendations in 3s5z_vs_3s6z map at a timestep (Centralized perspective).

actions are then combined to form the joint action a_t for all agents. This method reduces the computational load per call, as each prompt contains less information. However, it requires multiple LLM calls—one for each agent—leading to higher overall latency but offering better scalability for environments with many agents.

In both approaches, the LLM's textual suggestions are then converted into executable commands that the environment API can understand. This ensures that the agents can act on the LLM's recommendations, driving the system forward effectively (see Fig. 4).

3.3. Multi agent reinforcement learning based on LLM-assisted decision making

In this section, we describe how LLM-generated strategies are integrated into the MARL framework to improve the effectiveness of exploration and accelerate the training process.

During the initial training phases, the exploration strategy selects between LLM suggestions and random actions based on a probability ϵ_t at each time step t . Initially, ϵ_t is set to a high value. As training progresses, ϵ_t decays, gradually reducing the reliance on LLM suggestions and increasing the focus on RL-based exploration.

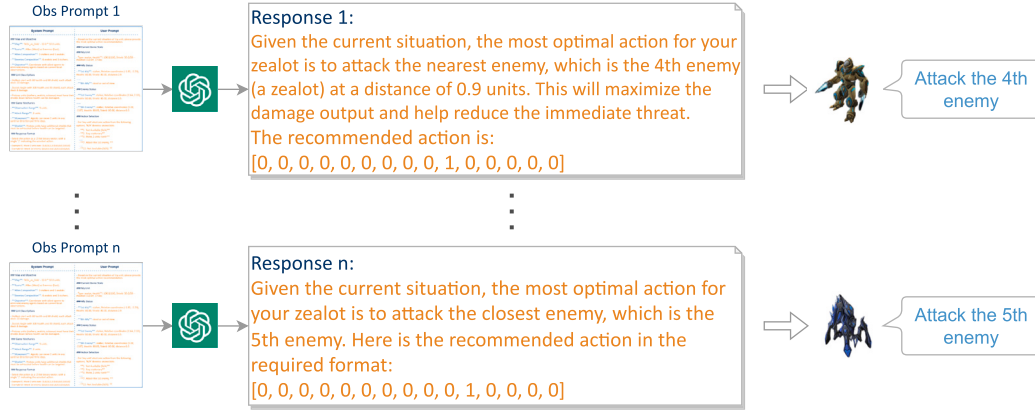


Fig. 4. Action recommendations in 3s5z_vs_3s6z map at a timestep (Decentralized perspective).

The replay buffer D_{LLM} initially contains trajectories obtained through both random exploration and LLM suggestions, including $(s_t, a_t^{LLM}, r_t, s_{t+1})$ and $(s_t, a_t^{random}, r_t, s_{t+1})$. The contributions from the LLM are not limited to fully LLM-guided trajectories; they also encompass action suggestions from the LLM encountered during random exploration states. As training progresses, D_{LLM} incorporates additional trajectories resulting from decisions made by the policy network.

The policy function is denoted as $\pi_\theta(a|s)$, where θ represents the parameters of all agents in the MARL network. To initialize the policy network, we maximize the likelihood of the data sampled from D_{LLM} :

$$\theta_0 = \arg \max_{\theta} \mathbb{E}_{(s,a) \sim D_{LLM}} [\log \pi_\theta(a|s)].$$

This initialization ensures that the policy network can effectively emulate the strategies provided by the LLM at the beginning of training.

Following the initialization, the policy network is fine-tuned using a composite loss function that balances the contributions of LLM-generated data and real-world experience:

$$L(\theta) = \lambda \cdot \mathbb{E}_{(s,a) \sim D_{LLM}} [-\log \pi_\theta(a|s)] + (1 - \lambda) \cdot \mathbb{E}_{(s,a) \sim D_{env}} [-\log \pi_\theta(a|s)],$$

where λ is a hyperparameter that controls the relative weight of the two data sources. Initially, λ is set to a higher value to prioritize LLM-generated data, and it gradually decreases as the agents gather more experience from the environment. This allows the policy network to refine its understanding of the environment, reducing dependency on LLM suggestions and focusing more on data from the actual environment. Over time, the agents' policies become more robust and better suited to the specific challenges of the multi-agent system.

4. Toolkit presentation

In this section, A toolkit has been designed to assist researchers grappling with the cold-start challenge in MARL, alongside those exploring the interplay among MARL, LLMs, and the SMAC environment. Since communication among agents is critical for coordination, the toolkit also offers functionalities for communication algorithm training, robust communication testing, and a visualization interface for communication algorithm analysis tailored for real-world multi-agent communication.

Section 4.1 outlines the process of integrating LLMs with MARL algorithms, exemplified by QMIX as shown in Fig. 5(e). Section 4.2 delves into the training, testing, and visualization of communication algorithms, with its visualization tools depicted in Fig. 5(d). Section 4.3 examines the interplay between SMAC, LLMs, and MARL, illustrated in Fig. 5(c). Section 4.4 details the plug-and-play functionalities highlighted in Fig. 5(a) and (b), ensuring users can navigate through different aspects of the toolkit to understand how each component addresses complex challenges in multi-agent systems.

4.1. LLM-assisted MARL training

This section introduces the primary module designed to enhance MARL through the use of LLMs. Key features of this module include the supported environments, the selectable models, and detailed guidance on usage. This section provides an overview of these functionalities to help users effectively utilize the toolkit.

4.1.1. Supported environments

The module supports the SMAC environment, which is widely used for MARL research. Users have the flexibility to choose from a variety of pre-defined SMAC maps or create their own custom maps using the StarCraft map editor. This versatility allows researchers and practitioners to tailor the training scenarios to their specific needs.

4.1.2. Available algorithms

The training methods supported by the module include fully decentralized, fully centralized, and centralized training with decentralized execution (CTDE). Users can choose the most appropriate method according to the specific scenario requirements of their task.

In the fully decentralized model, each agent operates independently, similar to traditional single-agent reinforcement learning, which is suitable for LLM-assisted training in the decentralized perspective. In contrast, the fully centralized model employs a central controller that gathers information from all agents and makes collective decisions. It is suitable for LLM-assisted training in the centralized perspective.

The CTDE approach is suitable for most tasks targeting the SMAC environment due to its complexity, especially for maps with difficulty levels classified as “hard” and above. Whether to utilize LLM-supported training from the decentralized or centralized perspective should be based on the specific requirements and objectives of the user.

4.1.3. Usage method

To effectively utilize the toolkit for MARL tasks, follow these steps:

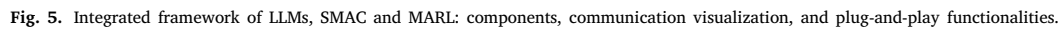
1. Environment Setup and Package Installation:

• System Requirements:

- Operating System: Linux or macOS.
- Python Version: Python 3.8 or higher.

• Dependencies Installation:

- Clone the repository from GitHub.
- Run the `install_sc2.sh` script provided in the repository to install StarCraft II and its dependencies. This script sets up the necessary environment for SMAC.



- Navigate to the directory containing the training scripts.
- Adjust the agents files and the controllers files to configure the desired LLM mode (centralized or decentralized).
- Execute the training script with appropriate parameters:

```
python run.py --env=sc2  
--alg=<selected_algorithm>  
--map=<map>.
```
- Adjust other hyperparameters in the file `config.yaml`.

3. Results Viewing:

- Logs and results are automatically saved in the `results` directory.

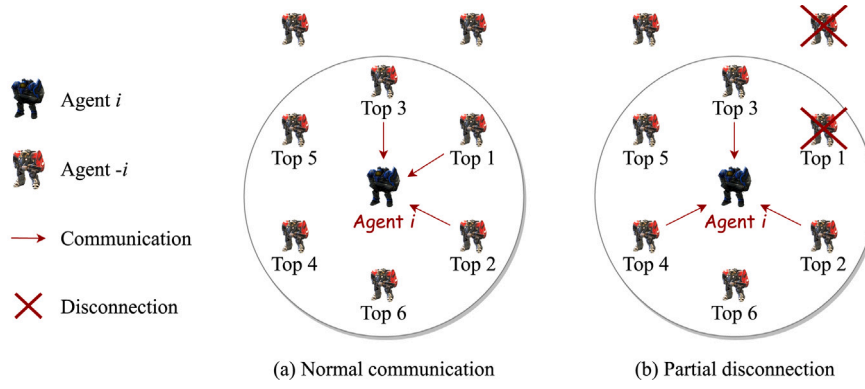


Fig. 6. Communication adjustments when agents lose connection.

- The training process logs, including early decision-making processes by LLMs, can be monitored via the `cout.txt` file in the `results` folder.
- The performance metrics such as win rates and returns are stored in an `info.json` file in the `results` folder. This file contains essential information for performance assessment and comparison.

By following these steps, users can efficiently use the toolkit to meet their specific needs and improve the performance of their multi-agent systems.

4.2. Robust communication algorithm

Given the importance of communication algorithms in MARL, we have provided a module adapted to real-world communication environments. This module includes the workflow for communication algorithm training, testing, and visualization.

4.2.1. Robust communication algorithm training and testing

In real-world scenarios, communication efficiency is often hampered by limited bandwidth. To address this, we have developed a one-to-one communication framework that can adaptively select the communication partners for each agent based on the current environmental conditions, including communication rates. This framework is also equipped to handle unexpected situations, such as sudden disconnections of some agents or reductions in communication rates due to external interference. We have integrated this communication mechanism into the CTDE framework, using the QMIX algorithm as a specific implementation example.

Consider a multi-agent system operating under bandwidth constraints, where each agent is permitted to communicate with a fixed number of other agents within its communication range based on the communication rates dictated by the environment. Before communication, each agent feeds its local observations into a neural network to generate an importance distribution. Based on this distribution, each agent performs importance sampling to determine the importance weights of potential communication partners. These weights are then ranked, and the final selection of communication partners is made considering the communication rate and the ranking.

During communication, each agent receives information from the selected agents. After communication, each agent updates its action-value function, or Q-value, by integrating its local observations with the received information. The individual Q-values are then aggregated to a total Q-value by a monotonic integration network. The communication and the value function modules are trained using the DQN methodology.

To further demonstrate the robustness of our communication algorithm, we conducted tests by introducing random disconnections

between agents during the testing process. Specifically, at any given time step, a predefined proportion of agents were randomly disconnected and their communication capabilities were disabled. This setup allowed us to evaluate the adaptability and robustness of the system under such unexpected scenarios. As visualized in Fig. 6. Initially, in Fig. 6(a), each agent can communicate with up to three allies, and the algorithm selects the top three as communication partners. When two agents are randomly disconnected, as shown in Fig. 6(b), the algorithm dynamically adjusts the communication partners.

4.2.2. Robust communication visualization

This visualization tool simplifies the analysis of communication processes, helping users refine their communication algorithms. It shows how different agents communicate at each time step during a battle. Users can select specific time steps and agents to see if an agent is communicating at that time and which other agents it is connected to. Key features include:

1. It shows the live positions and conditions of agents while updating their communication statuses in real time. Users can also view interactions with other agents based on selected agents and time steps.
2. The tool allows pausing and resuming animations during playback. It also offers the ability to save visual snapshots of specific stages.
3. It provides clear insight into the movement paths and communication activities of agents, helping to better understand their interaction strategies and communication patterns more effectively.

To demonstrate the tool's functionality, consider an example where we want to inspect Agent 6's communication status at the 23rd time step of the 10th episode. We use the command: `python visualization.py --selected_agent 6 --episode 10 --time_step 23`.

Executing this command generates a visualization, shown in Fig. 7, which provides a detailed snapshot of Agent 6's position, its connections with other agents, and overall communication activity at that time step. This example shows how the visualization tool can be effectively used to analyze and improve communication strategies in multi-agent systems.

4.3. Interactions among SMAC environment, LLMs, and MARL algorithms

This section describes the critical role of the toolkit in facilitating full-process interactions among the SMAC environment, LLMs, and MARL algorithms.

1. **Data Exchange and Transformation:** The toolkit converts code-formed information from the SMAC environment into natural language prompts suitable for LLM processing and transforms LLM-generated action suggestions back into a format executable by agents. This ensures seamless data flow between components.

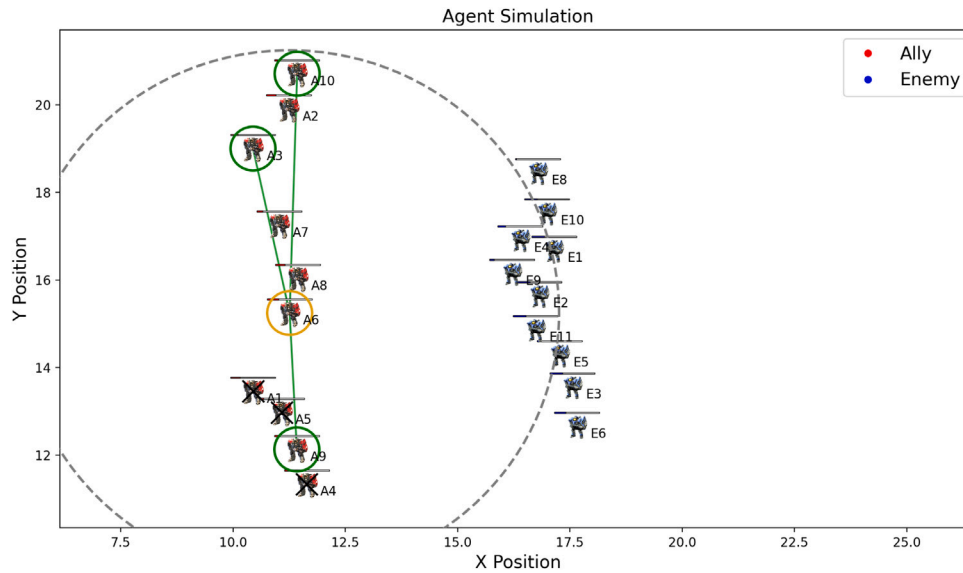


Fig. 7. Visualization of agent 6's communication status at the 23rd time step of the 10th episode.

2. **Policy Formulation and Optimization:** By integrating guidance from LLMs with real-world experience gathered by agents, the toolkit supports the generation of initial policies and their continuous refinement based on feedback. This process helps evolve strategies to better handle complex tasks.
3. **Feedback Loop and Dynamic Adjustment:** The toolkit manages the complete feedback loop, from agents executing actions to receiving feedback and updating policies. It automatically collects and processes trajectory data, providing essential inputs for MARL algorithms to dynamically adjust strategies.
4. **Full-Process Support:** The toolkit can be flexibly embedded across decision-making, training, and evaluation phases, promoting synergy between stages and simplifying cross-component data handling. This improves overall system performance and efficiency.

Through these functionalities, the toolkit not only fosters effective collaboration among components but also accelerates learning and improves strategy quality. This makes the system more adept at addressing complex multi-agent tasks. The next section details the toolkit's specific functions.

4.4. Plug-and-play functionalities

This section outlines the core functionalities of the toolkit, emphasizing their modular design and ease of integration with users' algorithms. Each functionality—LLM Selection, Prompt Generation, Strategy Alignment, and Visualization—is designed to be flexible and easily adaptable. Users can integrate these components into their workflows seamlessly, optimizing the performance of their multi-agent systems. The modular architecture allows for components to be added or removed based on specific needs, ensuring flexibility and user-friendliness.

4.4.1. LLM selection

The toolkit supports the selection of the most suitable LLM for your specific multi-agent scenario. This process involves evaluating a few LLMs using your algorithm and environment to determine which model yields the highest rewards, win rates, and lowest invocation time.

1. **Interface:** The toolkit provides a standardized interface with two core components:

- **Model Registry:** Central configuration storing all candidate LLMs in a structured format. The registry's `llm_registry` field contains key-value pairs mapping model names to their specifications, as shown in Fig. 8(a).
- **Model Invocation:** Execute model inference through the unified interface in Fig. 8(b).

2. LLM evaluation:

- The toolkit evaluates each candidate LLM by running it through a small set of tests within your specified scenario. It measures performance based on total rewards, win rates, and invocation time over a limited number of episodes.
- For some predefined maps, the toolkit already includes test results that users can refer to for initial guidance.
- If you are using custom LLMs or maps not covered in the pre-existing tests, the toolkit allows you to conduct your own evaluations. Simply input your custom LLM or map into the toolkit, and it will generate performance metrics for your specific setup.
- Based on these metrics, the toolkit identifies the LLM with the highest performance and recommends it for subsequent tasks.

4.4.2. Prompt generation

The core function of the toolkit is to convert information obtained via the SMAC API—whether its tensors representing an individual agent's local observations (in a decentralized perspective) or tensors representing the global environmental state (in a centralized perspective)—into natural language prompts. This enables large language models (LLMs) to make decisions or understand complex systems based on these prompts. The functionality is designed to be highly modular, allowing users to flexibly select configurations according to their specific needs.

To independently use this functionality, users should consider the following points:

1. Data Input

Choose the appropriate input data based on the application scenario:

- **Decentralized Perspective:** Inputs are tensors representing an individual agent's local observations, retrieved from the SMAC API. Suitable for scenarios where personalized action recommendations for each agent are required.

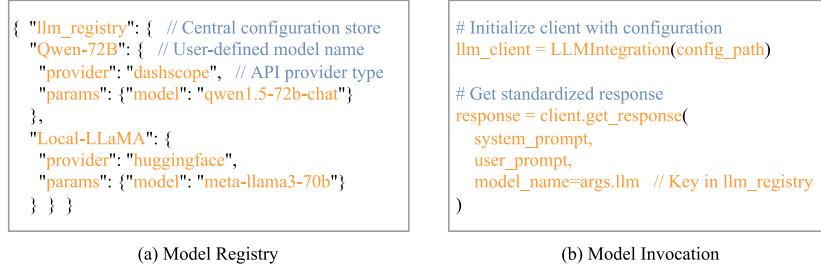


Fig. 8. LLM integration paradigm: configuration and execution.

- Centralized Perspective: Inputs are tensors representing the global environmental state, also sourced from the SMAC API. Ideal for situations where coordinated actions for all agents are needed by analyzing the entire environmental state.

2. Data Configuration

The toolkit offers multiple configuration options to meet diverse requirements:

- It supports sequential local observations or batched global states, and processes single-environment or parallel multi-environment data for scalable integration.
- Users can choose to generate natural language prompts for further analysis or directly convert them into executable commands.

3. System Integration

This functionality is designed for easy integration with other system components, whether for optimizing individual agents or coordinating actions across the entire system:

- Outputs can be directly fed into LLMs for processing, obtaining personalized action recommendations or team coordination strategies.
- It supports quick parsing of LLM outputs and converting them into actionable commands.

By following these steps, users can freely combine and utilize these functional modules based on their specific needs, leading to more efficient achievement of goals.

5. Experiments

In this section, we conduct experiments to achieve two main goals: (1) evaluate the performance of various LLMs in the SMAC environment, and (2) assess the effectiveness of our LLM-assisted decision-making approach in addressing the cold start problem. Through these experiments, we aim to provide insight into the capabilities of different LLMs and the effectiveness of our solution in overcoming initial performance challenges.

5.1. Environment settings

The experimental evaluation of our communication strategies in the MARL framework was conducted within the StarCraft Multi-Agent Challenge (SMAC). This renowned environment simulates real-time strategy game scenarios from StarCraft II. SMAC presents a series of micro-battles, each with unique parameters such as troop size, unit types, and initial positions, creating diverse challenges. We focused on a team of controllable allied agents, programmed to learn through MARL, against enemy units controlled by the game's built-in AI. The primary goal was for our agents to inflict more damage on adversaries while minimizing self-inflicted casualties, ultimately leading to higher average win rates.

Table 1

Overview of test maps.

Map	Difficulty	Allies	Enemies
1c3s5z	Very hard	1 Colossus, 3 Stalkers, 5 Zealots	1 Colossus, 3 Stalkers, 5 Zealots
5m_vs_6m	Very hard	5 Marines	6 Marines
8m_vs_9m	Very hard	8 Marines	9 Marines
10m_vs_11m	Very hard	10 Marines	11 Marines
3s5z_vs_3s6z	Very hard	3 Stalkers, 5 Zealots	3 Stalkers, 6 Zealots
MMM2	Very hard	1 Medivac, 2 Marauders, 7 Marines	1 Medivac, 3 Marauders, 8 Marines
Corridor	Very hard	6 Zealots	24 Zerglings
6h_vs_8z	Very hard	6 Hydralisks	8 Zealots

To thoroughly test and compare the efficacy of our algorithm, we selected eight maps from the difficult complexity level in the SMAC, including 1c3s5z, 5m_vs_6m, 8m_vs_9m, 10m_vs_11m, and 3s5z_vs_3s6z, MMM2, corridor and 6h_vs_8z, as shown in Table 1. These scenarios offer a diverse range of combat dynamics. Notably, most of these maps are asymmetric, with our agents often outnumbered and involving homogeneous and heterogeneous maps. This complexity requires more sophisticated tactics, making the cold start phase particularly challenging.

5.2. SMAC combat performance with different LLMs

5.2.1. Comparative LLMs

To investigate the combat performance of various LLMs in the SMAC environment, we conducted the experiments using eight different LLM models: Qwen2.5-72b, Qwen-max, LLaMA3-8b, LLaMA3-70b, GLM-4, GPT-4, DeepSeek-r1-8b, and DeepSeek-r1-70b.

Qwen2.5-72b is an advanced large-scale transformer-based language model with 72 billion parameters [42]. It is known for its strong capabilities in understanding complex prompts and generating high-quality responses.

Qwen-max extends the Qwen family with enhanced optimization for multi-task settings and cross-domain adaptability [42], making it a strong baseline for strategy generation tasks.

LLaMA3-8b is a lightweight version of the third-generation LLaMA models with 8 billion parameters [43], designed for efficient processing while maintaining competitive performance in dialogue-based tasks.

LLaMA3-70b is a larger variant of LLaMA-3, with 70 billion parameters [43], capable of handling more complex and nuanced interactions due to its richer representation capacity.

GLM-4 is the fourth-generation General Language Model [44], which introduces multi-modal understanding and advanced fine-tuning capabilities, making it well-suited for our prompt-based interaction experiments.

GPT-4 is an industry-standard large language model with exceptional performance in natural language processing and generation tasks [45], serving as a strong reference for high-quality intelligent strategy generation.

Table 2
Average battle rewards from decentralized and centralized perspectives.

	Map	LLM								
		Random	Qwen2.5-72b	Qwen-max	LlaMA3-8b	LlaMA3-70b	GLM-4	GPT-4	DeepSeek-r1-8b	DeepSeek-r1-70b
Decentralized	1c3s5z	4.7447	3.0468	9.8963	2.0118	5.3596	9.2406	9.8957	6.0682	11.4679
	5m_vs_6m	1.3302	5.9245	3.2075	0.0000	1.5849	3.6604	4.7925	0.4528	3.4340
	8m_vs_9m	1.5755	6.3309	3.6835	0.1727	4.2014	3.3094	7.7122	0.8633	5.6547
	10m_vs_11m	1.7516	6.1739	3.4534	0.0000	2.7081	2.8323	6.8593	1.0434	3.4534
	3s5z_vs_3s6z	2.7650	7.0659	7.9536	0.0000	3.5000	6.8997	6.3234	0.8458	6.2620
	MMM2	1.6296	5.2476	2.9765	0.0000	0.7926	4.8277	5.7651	1.3719	2.2368
	corridor	1.9459	6.0151	1.4063	0.0000	0.0000	5.5160	6.0235	0.8071	2.5441
	6h_vs_8z	4.6757	7.8649	7.4459	0.1622	6.8108	7.2162	7.6216	1.5541	6.6727
	Average	2.5523	5.9587	5.0029	0.2933	3.1197	5.4378	6.8742	1.6258	5.2161
Centralized	1c3s5z	4.7447	5.1404	10.6791	2.3596	5.3743	9.9759	10.9198	5.9479	2.2527
	5m_vs_6 m	1.3302	4.1132	4.7925	0.2264	1.8113	4.1132	6.4151	0.2264	1.3585
	8m_vs_9 m	1.5755	1.8993	6.8201	0.1727	1.5540	6.6475	8.4317	1.2086	1.3813
	10m_vs_11 m	1.7516	0.7453	6.6087	0.4472	0.8944	7.8261	7.2795	0.1490	1.1925
	3s5z_vs_3s6z	2.7650	5.0060	8.4716	1.6033	4.9341	6.7964	8.8503	2.5284	1.9132
	MMM2	1.6296	3.8636	4.5745	0.4565	0.3237	4.6021	5.7695	0.4565	0.7468
	corridor	1.9459	3.9488	5.9546	0.0000	0.0000	5.9118	7.5892	0.3974	1.8009
	6h_vs_8z	4.6757	2.6622	5.1892	4.3919	4.9054	5.5337	6.3919	1.7297	1.3581
	Average	2.5523	3.4224	6.6363	1.2072	2.4747	6.4259	7.7059	1.5805	1.5005

DeepSeek-r1-8b is a knowledge-distilled 8-billion-parameter variant from the DeepSeek-r1 series [46], optimized for efficient inference through layer reduction and quantization-aware training techniques.

DeepSeek-r1-70b utilizes hierarchical knowledge distillation in its 70-billion-parameter architecture [46], preserving core reasoning capabilities while reducing computational demands through attention mechanism optimizations.

5.2.2. Implementation details

Our objective was to measure the reward values generated during battles using both centralized and decentralized perspectives. This dual-perspective approach helps us understand how each LLM performs under conditions requiring global coordination or individual adaptability.

In the centralized mode, the LLM was provided with the global state of the environment at each time step and tasked with generating a coordinated action plan for all agents. In the decentralized mode, each agent fed its local observation to the LLM independently, which then produced an action based solely on that agent's perspective.

We ran each experiment for a fixed number of episodes, with 8 parallel environments generating 8 episodes simultaneously. All LLMs utilized their respective API default inference parameters as officially documented by the model providers. The reward value was calculated as the sum of immediate rewards received by the agents over the course of an episode. We computed the mean reward value across the 8 episodes to obtain the final results.

5.2.3. Overall performance

The overall performance results are summarized in Table 2. We evaluated the combat effectiveness of several large language models (LLMs) across multiple maps using both decentralized and centralized approaches. The first column shows the average reward obtained during random exploration. Most models, with the exception of the LLaMA and DeepSeek series, achieved higher average rewards than random exploration, demonstrating their ability to learn effective strategies in these environments.

In the decentralized approach, GPT-4 achieved the highest average rewards across most maps. Among the open source models, Qwen2.5-72b performed second best, followed by the proprietary models Qwen-max and GLM-4. The LLaMA series models showed moderate performance, with the smaller model LLaMA3-8b achieving lower rewards than those from random exploration.

Notably, on the 3s5z_vs_3s6z map, Qwen2.5-72b, Qwen-max, and GLM-4 outperformed GPT-4 by approximately 0.74, 1.63, and 0.58

points, respectively. In addition, Qwen2.5-72b outperformed GPT-4 on the 5m_vs_6 m and 6h_vs_8z maps, highlighting its superior performance on these complex scenarios in a decentralized setup. DeepSeek-r1-70b exhibits a striking capability dichotomy: it achieves the highest decentralized reward (11.4679) on the 1c3s5z map, yet suffers the most severe performance degradation in centralized settings among all 70B-scale models.

In the centralized approach, GPT-4 again led with the highest average rewards on most maps, followed by Qwen-max and GLM-4. Open source models generally did not perform as well. Centralized setups require handling more extensive prompts, which require higher quality output. This leads to limitations in models such as Qwen2.5-72b and LLaMA3-70b, whose architectural biases favor localized decision trees over holistic state processing. DeepSeek-r1-70b's centralized performance (1.5005 average) registers nearly 70% lower than its decentralized results, suggesting fundamental incompatibility between its reasoning paradigm and global coordination requirements. As a result, GPT-4 consistently achieved the highest rewards on almost all maps, except for GLM-4 on the 10m_vs_11 m map, where it outperformed GPT-4 by about 0.55 points.

When comparing the two perspectives, the proprietary models showed higher average rewards in the more complex centralized setting. Specifically, GPT-4 and GLM-4 improved their performance by about 1 point, and Qwen-max improved by about 1.6 points. This contrast is magnified in DeepSeek-r1-70b, whose 70.3% performance drop from decentralized to centralized settings exceeds even Qwen2.5-72b's 42.6% decline. However, it is noteworthy that large open source models such as Qwen2.5-72b and LLaMA3-70b had lower average rewards in the centralized mode compared to the decentralized mode. In particular, Qwen2.5-72b had significantly higher average rewards (about 2.54 points) on several maps in the decentralized mode. Furthermore, except for the simpler 1c3s5z map, Qwen2.5-72b achieved higher rewards in decentralized mode than all other maps. This indicates that the high requirements of the centralized perspective pose a challenge to open-source models, even with large parameter sizes.

The observed performance differences may reflect inherent architectural biases in handling coordination paradigms: Qwen2.5-72b's hierarchical decision making technique may favor decentralized autonomy, explaining its higher reward score (5.9587 vs 3.4224). This architecture is advantageous for achieving efficient local decision making in complex environments. LLaMA3-70b's standard autoregressive design may lack effective mechanisms for action coordination, potentially contributing to its lower centralized mean reward (2.4747). This design may underperform in tasks requiring synchronous action by multiple agents. GPT-4's Mixture of Experts (MoE) architecture

Table 3
GPT-4 Hyperparameter configuration.

Hyperparameter	Value
Temperature (τ)	1.0 (full stochastic sampling)
Top-p sampling (p)	1.0 (no nucleus filtering)
Max tokens per response	2048
Frequency/presence penalty	0.0

could enable synergistic strategic-tactical processing, sustaining high centralized rewards (7.7059). Such an architecture efficiently allocates resources and tasks to optimize overall performance. DeepSeek-r1-70b's attenuated cross-agent attention layers may limit global optimization, resulting in a significant 69.8% decrease in centralized performance. This suggests that the model may not perform as well as competitors in tasks requiring extensive collaboration and information sharing among agents.

Experimental results show that some models excel in decentralized operations, while others perform better in centralized coordination. Open source models perform better in decentralized tasks, while proprietary models dominate in team coordination. The performance gap between decentralized and centralized approaches varies significantly across models, indicating the need to tailor models to specific multi-agent needs.

5.3. Cold start solutions with LLM-assisted decision-making

5.3.1. Implementation details

To assess how using a LLM can improve the initial learning phase of MARL, we conducted tests on four highly challenging maps from the SMAC environment. These maps are known for being particularly tough for the baseline algorithm QMIX to start efficiently. The selected maps are 5m_vs_6m, 8m_vs_9m, 10m_vs_11m, and 3s5z_vs_3s6z. In most scenarios, these maps involve situations where our agents face a numerical disadvantage, complicating strategy development. We deployed GPT-4 as the assisted LLM with default generation parameters as detailed in Table 3.

The LLM provided strategic guidance during 20% of the first 2000 exploration steps by injecting natural language tactical suggestions into the agents' action selection process. This intervention was phased out linearly after the initial guidance period to ensure autonomous policy convergence.

We compared the average win rates over the first 6 million training steps between two experimental conditions: Baseline QMIX with standard exploration, and QMIX augmented with LLM-guided exploration. All experiments used 8 parallel environments with synchronized episode execution.

5.3.2. Overall performance

The performance of the QMIX algorithm with and without LLM-assisted decision-making across four different maps is illustrated in Fig. 9. The results indicate that integrating LLM significantly accelerates the training efficiency of MARL.

In the 5m_vs_6m map, the average win rate began to rise sharply between 1 million and 2 million steps due to LLM assistance. By 3 million steps, there was a 15% increase in win rate compared to the standard QMIX algorithm. For the 8m_vs_9m and 10m_vs_11m maps, the win rate divergence occurred within the first 1 million steps, with both scenarios showing an approximate 50% improvement in win rate at this milestone.

On the most complex map, 3s5z_vs_3s6z, the standard QMIX algorithm achieved only a 10% win rate by 8 million steps. However, with LLM assistance, the win rate improved to 30% at the same step count.

These findings demonstrate the substantial benefits of integrating LLMs into MARL algorithms, improving both training efficiency and overall performance. The use of LLMs can significantly boost the effectiveness of multi-agent systems in complex environments.

6. Conclusion

Our research presents an approach for integrating large language models (LLMs) into multi-agent reinforcement learning (MARL), particularly in complex cooperative game environments. By leveraging LLMs, we provide strategic guidance during the early stages of training, which significantly mitigates the challenges of starting from scratch, this integration effectively addresses the cold-start problem by increasing the efficiency of the initial training phase and improving cooperation among agents.

A key contribution is the development of a multifunctional toolkit that seamlessly integrates with the StarCraft Multi-Agent Challenge (SMAC) environment's API. This toolkit supports dynamic prompt generation, automatic strategy conversion, and LLM integration, as well as communication visualization capabilities. These capabilities advance MARL research by providing robust support for various tasks, making it easier for researchers to explore new possibilities and optimize their experiments.

In addition, our toolkit emphasizes effective communication to improve collaboration efficiency. It includes functionality for training, testing, and visualizing communication algorithms tailored to real-world multi-agent scenarios. These tools provide deeper insight into agent communication and cooperation, enabling more efficient strategies in complex environments.

Our toolkit improves experimental flexibility and efficiency, allowing researchers to better understand complex multi-agent interactions and optimize overall system performance. The structured approach ensures that users can navigate through different aspects of the toolkit easily, applying each component to address specific challenges in multi-agent systems.

In summary, this study provides a promising direction for addressing the cold-start challenge in MARL and opens new research avenues, particularly in reducing the dependency on labeled data for supervised learning. These advancements hold significant potential for advancing the field of multi-agent systems, leading to more efficient, adaptable, and scalable solutions. Through these contributions, we hope to inspire further exploration and innovation in the domain of multi-agent reinforcement learning.

CRediT authorship contribution statement

Zheming Li: Writing – original draft, Methodology. **Ruobing Zhang:** Visualization. **Zhengming Wang:** Supervision. **Zheng Xie:** Supervision. **Yiping Song:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This paper is supported by National Natural Science Foundation of China (NSFC Grant No. 62106275) and Natural Science Foundation of Hunan Province, China (Grant No. 2022JJ40558).

Data availability

I have shared the link to my code in the abstract.

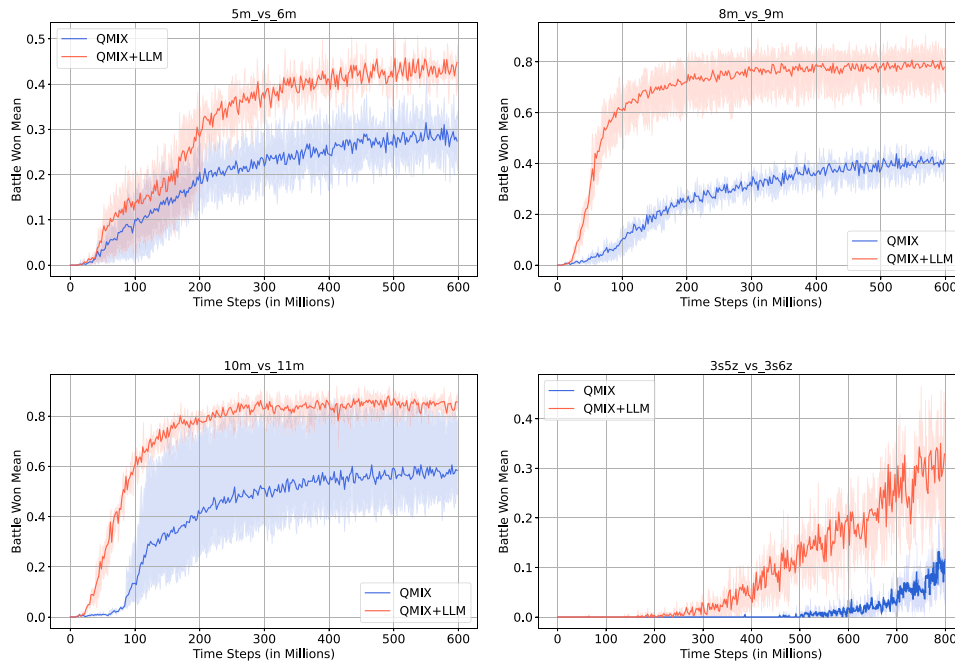


Fig. 9. Win rate comparison of QMIX algorithm with and without LLM-assisted decision-making.

References

- [1] P. Stone, G.A. Kaminka, J.S. Rosenschein, Ad hoc autonomous agent teams: Collaboration without pre-coordination, in: AAAI, 2010.
- [2] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in: NeurIPS, 2017, pp. 6379–6390.
- [3] J.N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, S. Whiteson, Counterfactual multi-agent policy gradients, in: AAAI, AAAI Press, 2018, pp. 2974–2982.
- [4] J.N. Foerster, Y.M. Assael, N. de Freitas, S. Whiteson, Learning to communicate with deep multi-agent reinforcement learning, in: NeurIPS, 2016, pp. 2137–2145.
- [5] O. Vinyals, I. Babuschkin, W.M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D.H. Choi, R. Powell, T. Ewalds, P. Georgiev, Grandmaster level in StarCraft II using multi-agent reinforcement learning, Nature 575 (7782) (2019) 350–354.
- [6] D. Pathak, P. Agrawal, A.A. Efros, T. Darrell, Curiosity-driven exploration by self-supervised prediction, in: CVPR Workshops, 2017, pp. 488–489.
- [7] P. Hernandez-Leal, M. Kaisers, T. Baarslag, E.M. de Cote, A survey of learning in multiagent environments: Dealing with non-stationarity, 2017, CoRR abs/1707.09183.
- [8] A. Tampuu, T. Matisen, D. Kodelja, I. Kuzovkin, K. Korjus, J. Aru, J. Aru, R. Vicente, Multiagent cooperation and competition with deep reinforcement learning, PLoS One 12 (4) (2017) e0172395.
- [9] T. Rashid, M. Samvelyan, C.S. de Witt, G. Farquhar, J.N. Foerster, S. Whiteson, QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning, in: ICML, 2018, pp. 4292–4301.
- [10] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489.
- [11] T. Hester, M. Vecerik, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, D. Horgan, J. Quan, A. Sendonaris, I. Osband, et al., Deep Q-learning from demonstrations, in: AAAI, 2018, pp. 3223–3230.
- [12] M.E. Taylor, P. Stone, Transfer learning for reinforcement learning domains: A survey, J. Mach. Learn. Res. 10 (2009) 1633–1685.
- [13] S. Agashe, Y. Fan, X.E. Wang, Evaluating multi-agent coordination abilities in large language models, 2023, CoRR abs/2310.03903.
- [14] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, in: NeurIPS, 2020.
- [15] M. Samvelyan, T. Rashid, C. Schroeder de Witt, G. Farquhar, N. Nardelli, T.G.J. Rudner, C.-M. Hung, P.H.S. Torr, J. Foerster, S. Whiteson, The StarCraft multi-agent challenge, 2019, CoRR abs/1902.04043. URL <https://arxiv.org/abs/1902.04043>.
- [16] O. Vinyals, T. Ewalds, S. Petrov, L. Sampedro, F. Huszar, L. Buesing, T. Bolukbasi, W. Dabney, StarCraft II: A new challenge for reinforcement learning, 2017, CoRR abs/1708.04782. URL <https://arxiv.org/abs/1708.04782>.
- [17] J. Hu, S. Jiang, S.A. Harding, H. Wu, S.-W. Liao, Rethinking the implementation tricks and monotonicity constraint in cooperative multi-agent reinforcement learning, 2021, arXiv:2102.03479.
- [18] P. Sunehag, G. Lever, A. Gruslys, W.M. Czarnecki, V.F. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J.Z. Leibo, K. Tuyls, et al., Value-decomposition networks for cooperative multi-agent learning based on team reward, in: AAMAS, 2018, pp. 2085–2087.
- [19] J.N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, S. Whiteson, Counterfactual multi-agent policy gradients, 2017, CoRR abs/1705.08926.
- [20] V. Mnih, A.P. Badia, M. Mirza, A. Graves, T.P. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: ICML, 2016, pp. 1928–1937.
- [21] K. Son, D. Kim, W.J. Kang, D. Hostallero, Y. Yi, QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning, in: ICML, 2019, pp. 5887–5896.
- [22] Y. Yang, J. Hao, B. Liao, K. Shao, G. Chen, W. Liu, H. Tang, Qatten: A general framework for cooperative multiagent reinforcement learning, 2020, CoRR abs/2002.03939.
- [23] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, N. de Freitas, Dueling network architectures for deep reinforcement learning, in: ICML, 2016, pp. 1995–2003.
- [24] P. Peng, Q. Yuan, Y. Wen, Y. Yang, Z. Tang, H. Long, J. Wang, Multiagent bidirectionally-coordinated nets for learning to play StarCraft combat games, 2017, CoRR abs/1703.10069.
- [25] T. Rashid, M. Samvelyan, C.S. de Witt, G. Farquhar, J.N. Foerster, S. Whiteson, Monotonic value function factorisation for deep multi-agent reinforcement learning, J. Mach. Learn. Res. 21 (2020) 1–51.
- [26] C.S. de Witt, T. Gupta, D. Makoviichuk, V. Makovychuk, P.H.S. Torr, M. Sun, S. Whiteson, Is independent learning all you need in the StarCraft multi-agent challenge? 2020, CoRR abs/2011.09533.
- [27] D. Horgan, J. Quan, D. Budden, G. Barth-Maron, M. Hessel, H. van Hasselt, D. Silver, Distributed prioritized experience replay, in: ICLR, 2018.
- [28] J.N. Foerster, N. Nardelli, G. Farquhar, T. Afouras, P.H.S. Torr, P. Kohli, S. Whiteson, Stabilising experience replay for deep multi-agent reinforcement learning, in: ICML, 2017, pp. 1146–1155.
- [29] C. Colas, T. Karch, O. Sigaud, P.-Y. Oudeyer, Autotelic agents with intrinsically motivated goal-conditioned reinforcement learning: A short survey, J. Artificial Intelligence Res. 74 (2022) 1159–1199.
- [30] D. Kim, S. Moon, D. Hostallero, W.J. Kang, T. Lee, K. Son, Y. Yi, Learning to schedule communication in multi-agent reinforcement learning, in: ICLR, 2019.
- [31] J. Jiang, Z. Lu, Learning attentional communication for multi-agent cooperation, in: NeurIPS, 2018, pp. 7265–7275.
- [32] A. Singh, T. Jain, S. Sukhbaatar, Learning when to communicate at scale in multiagent cooperative and competitive tasks, in: ICLR, 2019.
- [33] S.Q. Zhang, Q. Zhang, J. Lin, Efficient communication in multi-agent reinforcement learning via variance based control, in: NeurIPS, 2019, pp. 3230–3239.

- [34] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *Language models are unsupervised multitask learners*, 2019.
- [35] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, et al., Evaluating large language models trained on code, 2021, CoRR [abs/2107.03374](#).
- [36] G. Wang, G. Yang, Z. Du, L. Fan, X. Li, ClinicalGPT: Large language models finetuned with diverse medical data and comprehensive evaluation, 2023, CoRR [abs/2306.09968](#).
- [37] T. Gao, A. Fisch, D. Chen, Making pre-trained language models better few-shot learners, in: *ACL*, 2021, pp. 3816–3830.
- [38] H. Li, Y.Q. Chong, S. Stepputtis, J. Campbell, D. Hughes, C. Lewis, K.P. Sycara, Theory of mind for multi-agent collaboration via large language models, in: *EMNLP*, 2023, pp. 180–192.
- [39] Z. Yao, Z. Tang, J. Lou, P. Shen, W. Jia, VELO: A vector database-assisted cloud-edge collaborative LLM QoS optimization framework, in: *ICWS*, 2024, pp. 865–876.
- [40] C. Sun, S. Huang, D. Pompili, LLM-based multi-agent reinforcement learning: Current and future directions, 2024, CoRR [abs/2405.11106](#).
- [41] S.D. Morad, A. Shankar, J. Blumenkamp, A. Prorok, Language-conditioned offline RL for multi-robot navigation, 2024, CoRR [abs/2407.20164](#).
- [42] Qwen Team, Qwen technical report, 2023, URL https://qwen.readthedocs.io/zh_CN/latest/.
- [43] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, et al., Llama 2: Open foundation and fine-tuned chat models, 2023, arXiv preprint [arXiv:2302.13971](#).
- [44] Aohan Zeng, Xiao Liu, Zhengxiao Du, et al., GLM-4: General language model pretraining with autoregressive blank infilling, 2022, arXiv preprint [arXiv:2210.02414](#).
- [45] OpenAI, GPT-4 technical report, 2023, URL <https://cdn.openai.com/papers/gpt-4.pdf>.
- [46] DeepSeek Inc., DeepSeek-R1 technical documentation, 2024, URL <https://deepseek.com/docs>.